

ASSESSMENT 3

AI Procurement Request Copilot

Build a small, reliable AI product - then prove which architecture you would ship.

01 Client challenge

THE SITUATION

Employees request new software and services. Procurement must check existing tools, team budget, vendor status, security/privacy requirements and approval rules. Build an AI-powered internal product that gathers evidence and recommends the next action - while keeping sensitive decisions with humans.

STARTER PACK PROVIDED

DATA

Employees + budgets + software catalog
+ vendors + purchase history + requests

POLICY

Procurement rules + approval thresholds
+ security / privacy requirements

MOCK SERVICE

Vendor / security status endpoint for tool-
based retrieval

02 Expected workflow



AI retrieves evidence. Deterministic checks stay in code where appropriate.

03 What you must build

A PRODUCT UI	B TOOL USE	C AGENT SYSTEM	D EVALUATION
Request details Evidence panel Recommendation + action	At least 3 tools At least 1 deterministic tool	Working single-agent baseline Structured outputs + human handoff	Compare with a staged / 2-agent variant Use the same test set

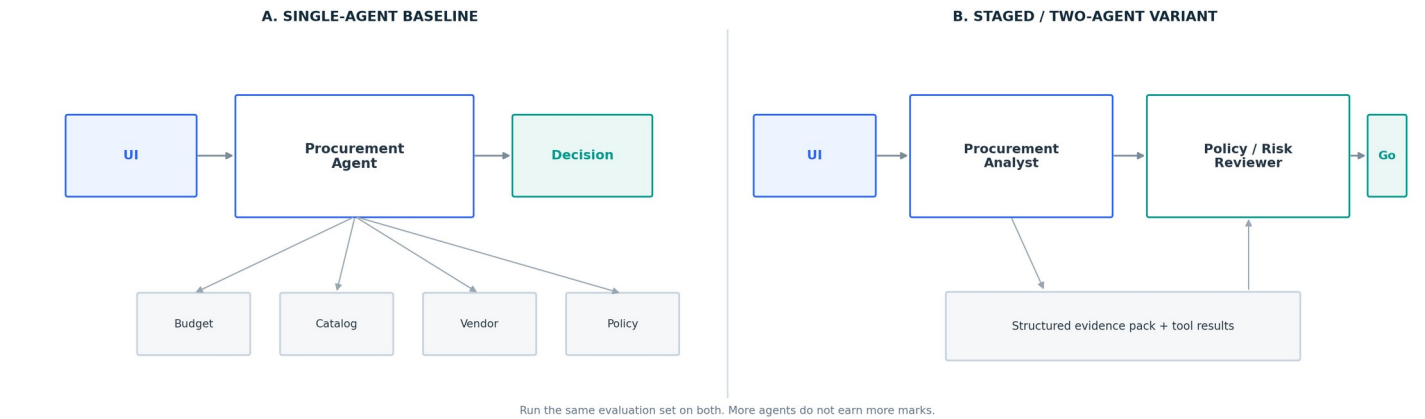
Required output: recommendation | evidence | approvals required | missing information | risk flags | next step

DESIGN PRINCIPLE

AI	CODE	HUMAN
Interpret context + recommend	Thresholds + deterministic checks	Sensitive approvals + exceptions

04 Architecture experiment

Build A first. Then implement a lightweight B and compare. Maximum recommended: 2 agents.



EVALUATE BOTH ARCHITECTURES

- Correct recommendation / next action
- Evidence is grounded in tool results
- Policy + deterministic rules are followed
- Escalation / human review is correct
- Latency + LLM/tool call count

EXPECT EDGE CASES

- Incomplete / ambiguous request
- Existing tool already solves the need
- Conflicting or expired vendor information
- Security-sensitive request or approval threshold
- Prompt injection inside business data
- Tool / API unavailable

05 Submission package

DELIVERABLES

- Git repository + clear README / setup
- Working local app with a one-command start path
- Architecture / workflow diagram + assumptions
- Reproducible evaluation script and results
- Architecture decision memo - max 500 words
- No secrets in the repo; include .env.example if needed

SCORING

Product + workflow	15%
End-to-end product	20%
Agent + tool design	20%
Reliability + human controls	15%
Evaluation + comparison	20%
Engineering + communication	10%

FINAL QUESTION

Which architecture would you ship - and why?

Use your evaluation evidence. A simpler system that performs as well or better is a stronger answer than unnecessary orchestration. UI polish and agent count do not earn extra marks by themselves.

BUILD Working vertical slice	EVALUATE Same test set on both	DEFEND Evidence-backed choice
--	--	---